PROJECT NEXUS Heterogeneous Compute Fabric

Engineering Architecture and Validation Specification | Revision 0.9 | October 16, 2026

Status: Preliminary design proposal. English edition. This document describes proposed components, constraints, and validation gates. It does not establish that every feature is implemented, universally compatible, or capable of meeting a particular performance target.

1. Executive Summary

Project Nexus proposes a compatibility and optimization platform for interactive applications. It begins with performance measurement, reversible configuration changes, and management of real resources. Later phases investigate bounded feature translation, heterogeneous task scheduling, and an attached mobile device as an explicit task worker or asset cache.

The platform does not create physical VRAM, instruction-set support, or graphics functionality by modifying capability reports. A capability may be advertised only when a real implementation exists and its required behavior has been validated. Correct translation, visual approximation, and unsupported functionality remain distinct categories.

Anti-cheat evasion is not a product feature. The original goal of safe operation is retained through authorized integration, explicit policies, isolation, and auditable behavior rather than stealth.

2. Objectives and Scope

2.1 Objectives

- Improve measured performance for explicitly supported applications and hardware configurations.

- Manage graphics resource residency and asset streaming against actual memory budgets.

- Provide validated alternative implementations for selected missing capabilities where feasible.

- Reduce rendering cost through supported settings or engine-aware integration.

- Evaluate independent tasks on integrated GPUs, accelerators, and connected phones.

- Expose changes, limitations, measurements, and rollback controls to the user.

2.2 Non-goals

- Guaranteed support for every AAA title on arbitrary below-minimum hardware.

- Increasing physical VRAM by changing adapter descriptions.

- Bypassing or hiding from anti-cheat systems.

- Treating generated frames as equivalent to newly rendered simulation frames.

- Implementing a complete graphics API emulator or universal CPU instruction translator in the MVP.

3. Design Principles

Principle	Engineering consequence
Truthful capabilities	Report native, translated, approximate, or unsupported status without fabricating physical resources.
Measurement first	Establish repeatable baselines before claiming an improvement.
Safe rollback	Back up configuration and retain a known local fallback.
Incremental delivery	Validate one integration and one workload before broadening scope.
Security and authorization	Use permitted interfaces; do not hide process modifications.
Transparency	Record decisions, evidence, constraints, and known failures.

4. Proposed Architecture

The MVP operates through external configuration and monitoring or through an integration inside an application or engine controlled by the project. A universal interceptor for closed-source games is not the starting assumption.

Component	Responsibility
User interface	Select supported targets, review changes, run benchmarks, and restore configuration.
Policy manager	Determine permitted operations for each target and integration mode.
Capability probe	Query actual CPU, GPU, memory, driver, and supported API capabilities.
Telemetry collector	Collect frame times, FPS, memory usage, and thermal data where available.
Resource planner	Manage cache capacity, prefetch, and eviction within measured budgets.
Quality controller	Recommend approved quality profiles against frame-time targets.
Task scheduler	Place eligible tasks using measured transfer, execution, and synchronization costs.
Mobile connector	Negotiate capabilities, dispatch tasks, enforce deadlines, and handle disconnects.
Execution supervisor	Validate recommendations and apply only authorized, bounded actions.
Audit log	Record changes, measurements, errors, and rollback results with data minimization.

5. Capability Registry and Feature Translation

Each capability has an explicit implementation status. API support queries are structured contracts, not generic Boolean values to override. An implementation must satisfy the relevant semantics before it is presented as compatible.

Status	Meaning	Activation rule
Native	Implemented by supported hardware and driver.	Use verified API queries and compatibility checks.
Translated	Mapped to supported operations while preserving required behavior.	Require correctness, compatibility, and performance validation.

Status	Meaning	Activation rule
Approximate	An intentional visual or numerical substitute.	Explicit opt-in and an integration that permits changed behavior.
Unsupported	No valid implementation or unacceptable cost.	Do not advertise support or force the path.

Registry fields: feature identifier, API version, detection method, available backends, semantic limitations, memory cost, measured runtime cost, correctness tests, tested hardware, and validation status.

Removing a reflection effect is a quality decision, not an equivalent implementation of ray tracing. Translating an operation correctly is also separate from making it fast enough for interactive use.

6. Memory and Asset Management

6.1 Memory model

Separate GPU-ready resident resources from stored assets. Monitor actual video-memory budgets and manage residency and synchronization through the applicable API. System memory, SSD storage, and phone storage are not interchangeable with local GPU memory.

6.2 Asset pipeline

Source assets -> asset index and usage prediction -> local or remote cache -> decompression and preparation -> GPU upload -> resident resources within budget. Critical state must remain recoverable locally; a phone disconnect must not destroy the only copy of required application state.

6.3 Proposed policies

- Enforce configurable RAM, SSD, and mobile-cache limits.

- Prefetch known assets only when an appropriate integration exposes asset identity and lifecycle.

- Evict unused resources at safe points and measure reloading stalls.

- Keep lower-detail alternatives available when supported by the engine.

- Record budget overruns and allocation failures rather than masking them.

- Exclude arbitrary page-fault interception and universal closed-source memory virtualization from the MVP.

Acceptance gate: reduce measured stalls or resource pressure without data corruption, uncontrolled RAM growth, or increased failure rates. A phone cache remains optional and must outperform an appropriate local baseline for its intended workload.

7. Missing Graphics Features and Legacy CPU Support

7.1 Bounded graphics translation

Start with one precisely specified feature. Define inputs, outputs, required semantics, supported target operations, memory requirements, and reference tests. Prefer an existing engine fallback when available. Reject unsupported behavior rather than converting a capability check into an unsupported promise.

7.2 AVX2 compatibility

- With source access or developer cooperation, provide alternative code paths and select them using verified CPU capabilities.

- Without source access, investigate dynamic binary translation as a separate, bounded research track.

- Validate arithmetic results, exception behavior, and architectural state for the supported scope.

- Measure translated-code coverage and whole-application overhead.

- Do not skip an instruction-set check without supplying a functioning alternative implementation.

Booting successfully is not sufficient evidence of correctness or playability. Functional compatibility and acceptable performance require separate acceptance tests.

8. Mobile Edge Node

Treat the phone as an explicit worker with independent memory, capability limits, transport overhead, and thermal constraints. Negotiate protocol version and supported workloads before dispatch.

Workload	Initial role	Validation gate
Asset cache	Store reloadable, noncritical assets.	Compare transfer and recovery time against local RAM/SSD alternatives.
AI inference	Run a particular model for an independent task.	Confirm actual backend and supported operators; measure end-to-end latency.
Upscaling or frame generation	Optional research pipeline.	Measure latency, artifacts, sustained thermals, and real versus generated frames.
Auxiliary compute	Process tasks with explicit input/output contracts.	Demonstrate net benefit after data transfer and synchronization.

Proposed latency budget: total task latency = preparation + outbound transfer + queueing + execution + return transfer + integration. Some stages may overlap, so measure actual end-to-end latency and sustained throughput rather than summing ideal component specifications.

Track dropped or stale results, thermal throttling, disconnect recovery, and processing cost on both devices. Hardware-accelerator availability does not imply that every operator or model runs there. Select a maintained mobile inference backend for the target devices; do not assume one API guarantees NPU execution.

Video compression can reduce image-transfer volume, but it does not bypass USB transport. Lossy video encoding is not a general substitute for preserving exact depth, motion, or numerical buffer values.

9. Rendering Degradation and Quality Control

9.1 Integration levels

- Level 1: documented, reversible configuration settings such as resolution, shadows, internal render scale, and frame caps.

- Level 2: an engine plugin with access to scene identity, resource dependencies, LOD, and approved shader alternatives.

- Level 3: a narrowly scoped external compatibility integration tested against specified application versions; no assumption of semantic knowledge from arbitrary draw calls.

9.2 Dynamic Quality Guardian

For a 30 FPS target, the nominal frame-time budget is approximately 33.3 ms. Performance degradation generally corresponds to frame times rising above this budget, not falling below it. Use a measurement window, hysteresis, minimum dwell time, and bounded quality profiles to avoid rapid oscillation.

9.3 Shader and geometry changes

Use tested, simplified shader variants instead of generic black or empty shader replacement. Perform culling from validated scene bounds and importance metadata exposed by an appropriate integration. A draw call alone does not establish object bounds, gameplay significance, or safe removal.

Acceptance gate: reference-scene comparisons show no unintended removal of gameplay-relevant objects, broken materials, or invalid resource dependencies. Document intentional visual differences.

10. Heterogeneous Task Scheduling

Each eligible task declares inputs, outputs, dependencies, deadlines, supported backends, transfer requirements, and a local fallback. Moving arbitrary dispatches to a different device is not sufficient: data accessibility and synchronization must be explicitly handled.

Proposed admission rule: transfer cost + remote execution cost + synchronization cost must be less than local execution cost with a measured safety margin. Also enforce correctness, resource limits, and deadline feasibility. Cross-adapter resource restrictions and synchronization requirements depend on the API and driver.

11. Disconnect Handling and Recovery

- Assign every task an identifier, data version, and deadline.

- Reject late, duplicated, or stale results before integrating them.

- Do not keep the sole copy of critical state on the mobile node.

- On disconnect, stop new dispatches and use a validated local fallback or disable the optional feature.

- Reconnect through fresh capability and protocol negotiation; do not resume blindly.

- Test disconnection during upload, execution, return, and reconnection.

Avoid promising sub-millisecond failover or unchanged performance without measurement. Distinguish application survival, state correctness, and preservation of frame rate as separate goals. Reducing graphical quality cannot restore a missing required resource by itself.

12. Security and Authorization

The original stealth objective is replaced by an explicit compatibility and authorization policy. Kernel drivers and hypervisors are not mechanisms for concealing modifications from anti-cheat systems.

Target environment	Default policy
Project-controlled application	Direct integration with full testing.
Application with permitted plugins	Use supported interfaces within documented permissions.
Protected multiplayer title	Only explicitly authorized functionality.
Unknown permission or protection state	No in-process modification by default.

- Verify update signatures and package integrity.

- Authenticate paired devices and protect transport as appropriate to the threat model.

- Validate message sizes, payload formats, and resource limits.

- Minimize logs, expose retention controls, and avoid unnecessary personal data.

- Consider a driver only for a specific legitimate requirement that cannot reasonably use suitable user-mode interfaces; require a dedicated security review.

13. Agent Roles and Deterministic Supervision

Agents propose bounded actions. A deterministic execution supervisor checks policies, budgets, preconditions, and rollback readiness before applying changes. A language model, if included, analyzes reports or proposes profiles for review; it does not directly control unrestricted process-memory edits or the per-frame safety path.

Agent	Responsibility
Capability agent	Collect actual support and limitations.
Resource agent	Recommend cache, prefetch, and eviction decisions.
Quality agent	Select among approved quality profiles.
Thermal and power guard	Constrain operation based on available telemetry.
Policy guard	Block unauthorized operations for the target application.
Execution supervisor	Validate, apply, reject, and log actions.

14. Measurement and Acceptance Framework

Area	Measurements
Performance	Average FPS, 1% lows with a documented calculation method, frame-time distribution, and input latency where measurable.
Memory	Usage, budget, residency pressure, allocation failures, and loading stalls.
Visual quality	Reference screenshots or sequences, known differences, and artifact checks.
Mobile execution	End-to-end latency, throughput, sustained thermals, deadline misses, and recovery outcomes.
Stability	Application crashes, device or driver errors, rollback success, and resource leaks.
Compatibility	Hardware, OS, driver, application/engine version, settings, and test workload.

Every performance claim must include baseline configuration, run count, measurement method, workload, and relevant uncertainty. Do not generalize a gain observed on one device or scene to all targets. Define numerical pass/fail thresholds for the selected workload before testing; none are asserted here as achieved results.

15. MVP Roadmap and Stage Gates

Phase	Deliverable	Exit gate
0: Test platform	Controlled application or scene; telemetry and baseline.	Repeatable results and configuration capture.
1: Local optimization	Approved profiles, frame-time monitoring, and rollback.	Measured benefit without correctness or stability regression.
2: Resource streaming	Local cache, budget policy, and safe eviction.	Pressure tests without loss of critical resources.
3: One translated feature	Bounded implementation and test suite.	Correctness, compatibility, and acceptable measured cost.
4: Local heterogeneous task	One independent workload on another local processor.	Net gain after transfer and synchronization.
5: Mobile connector	One noncritical task with fallback and recovery.	Validated operation under thermal load and disconnects.
6: Selected integrations	Authorized application-specific support.	Versioned compatibility and regression tests.

Legacy instruction translation remains a separate research track and must not block otherwise useful local optimization features.

16. Risk Register

Risk	Impact	Mitigation
VRAM budget exceeded	Stalls or allocation failure.	Budget monitoring, eviction, quality limits, and rollback.
Incorrect translation	Wrong output or visual corruption.	Reference tests and explicit unsupported status.
Remote execution overhead	Higher latency or lower throughput.	End-to-end measurement and local fallback.
Application updates	Broken or changed integration.	Versioned support and regression tests.
Phone thermal throttling	Missed deadlines and unstable performance.	Thermal limits and workload reduction.
Unauthorized integration	Policy violation or account sanctions.	Authorized interfaces and conservative defaults.
Quality oscillation	Distracting repeated transitions.	Hysteresis, dwell time, and approved profiles.

17. Open Decisions Before Implementation

- Select the first controlled application or engine integration.

- Choose the initial operating system and graphics API.

- Identify reference hardware and the first target bottleneck.

- Select one MVP success objective: frame-time consistency, memory pressure reduction, or proof of an independent offload task.

- Select a phone and transport setup for the first mobile experiment.

- Define licensing, supported integration policy, and version support commitments.

- Specify quantitative acceptance thresholds for the initial benchmark suite.

18. Conclusion

The revised design preserves the research goals of resource management, bounded capability translation, rendering simplification, heterogeneous compute, and mobile assistance. Each becomes a measurable implementation track rather than a universal promise. The recommended starting point is a controlled application, one performance objective, and a reproducible benchmark. Subsequent features advance only after correctness, performance, stability, and authorization gates are satisfied.

Technical Reference Basis

The following references support the API constraints and background discussed in the preceding conversation. Proposed architecture decisions and stage gates are design recommendations, not findings established by these sources.

Microsoft Learn: Hardware Feature Levels
https://learn.microsoft.com/en-us/windows/win32/direct3d12/hardware-feature-levels

Microsoft Learn: Residency
https://learn.microsoft.com/en-us/windows/win32/direct3d12/residency

Microsoft Learn: Memory Management Strategies
https://learn.microsoft.com/en-us/windows/win32/direct3d12/memory-management-strategies

Microsoft Learn: Shared Heaps
https://learn.microsoft.com/en-us/windows/win32/direct3d12/shared-heaps

Microsoft Learn: DrawIndexedInstanced
https://learn.microsoft.com/en-us/windows/win32/api/d3d12/nf-d3d12-id3d12graphicscommandlist-drawindexedinstanced

Android Developers: Neural Networks API
https://developer.android.com/ndk/guides/neuralnetworks

Android Open Source Project: NNAPI Runtime
https://source.android.com/docs/core/ota/modular-system/nnapi

Dorner and Klausner: A Critical Examination of Kernel-Level Anti-Cheat Systems
https://arxiv.org/abs/2408.00500